

OPEN SOURCE · PYTHON · FLASK

EduSync

Automating classroom attendance with real-time facial recognition.

A Flask + InsightFace system that detects every face in a classroom photo, matches it against stored embeddings, and logs attendance in seconds — no roll-call, no proxies, no manual Excel.

SCRFD · 2.5g

MobileFaceNet · 512-D

cosine $\geq$ 0.45

THE STATUS QUO

Manual attendance is broken at scale.

Roll-call, sign-in sheets, and biometric fingerprint scanners all fail the same way: they consume lecture time, leak to proxy attendance, and produce data that lives in disconnected Excel files.



5–10 min lost per lecture

Roll-call eats the opening of every class, compounding across an academic year.



Proxy attendance

Buddy-signing and forged signatures remain trivial to execute and hard to prove.



Disconnected data

Attendance lives in faculty-owned spreadsheets, not in any queryable system.



No scale

Manual tracking collapses across batches, time slots, and parallel sections.



A typical 80-seat classroom – every face is an attendance row.

One classroom photo becomes a complete attendance record.

A teacher uploads a single image of the room. EduSync tiles it, detects every face with SCRFD, embeds each face into a 512-D vector via MobileFaceNet, and cosine-matches against enrolled students — writing attendance rows for every match above 0.45 in under a second per face.

01

CAPTURE

Teacher uploads a single photo of the classroom from any device. No special camera, no per-student enrollment kiosk needed.



02

MATCH

Each detected face is embedded and cosine-matched against the student_face table in MySQL. Threshold ≥ 0.45 = present.



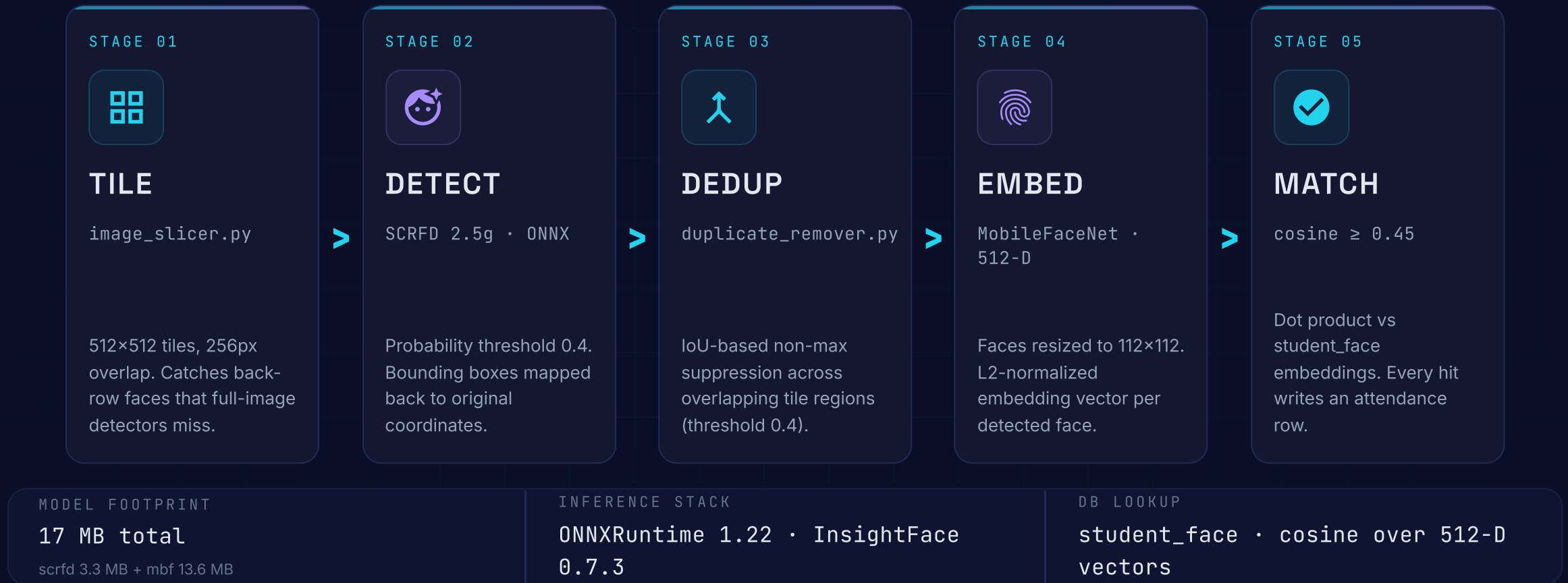
03

LOG

Attendance rows are written for every match with a unique (enrollment, date, time_slot) key. Faculty and mentor dashboards update instantly.

From a raw classroom JPEG to attendance rows.

Five stages. Two ONNX models. No cloud calls.



A pragmatic Flask + ONNX stack. Self-hostable on a single VM.

No proprietary cloud APIs. No GPU required. Both ONNX models ship in the repo.

BACKEND

Flask 3.1.1 Werkzeug 3.1.3
Jinja2 3.1.6 Gunicorn 23.0.0

Blueprint-per-role architecture (auth, admin, teacher, mentor, student, register, forgot, login).

AI / VISION

InsightFace 0.7.3 ONNXRuntime 1.22
OpenCV 4.12 NumPy 1.26

SCRFD for detection, MobileFaceNet for embeddings — both committed as ONNX.

DATABASE

MySQL 8 PyMySQL 9.4

5 tables: users, students, student_face, faculty, attendance. Embeddings stored as LONGBLOB.

FRONTEND

HTML5 CSS3 Vanilla JS jQuery
Bootstrap Font Awesome

Server-rendered Jinja2 templates with role-specific CSS and JS bundles. No build step, no SPA framework — fast to ship and easy to host.

INFRA

python-dotenv smtpLib (Gmail SMTP)

OTP password reset + faculty credential emails styled in HTML.

Layered Flask app — blueprints, services, models, ONNX.

Each role owns its blueprint. AI lives in `models/`. Mail lives in `service/`. State lives in MySQL.

BROWSER

Client

auth.html

login.html

admin.html

teacher.html

mentor.html

student.html

role-scoped Jinja2 templates + vanilla JS



FLASK

Application

app.py

routes/auth

routes/admin

routes/teacher

routes/mentor

routes/student

routes/forgot

8 blueprints registered at app startup



SERVICE LAYER

Services & AI

service/mail_service.py

models/detect_face.py

models/generate_embeddings.py

models/image_slicer.py

models/duplicate_remover.py

models/read_excel.py

pure Python — no network calls



INFRA

Models & Data

scrfd_2.5g_bnkps.onnx

w600k_mbf.onnx

MySQL · edusync

ONNX models ship in-repo (~17 MB); MySQL holds embeddings + attendance

Beyond face recognition — a full role-based management system.

Four dashboards, automated email, smart bulk import, and OTP password reset — all in one Flask app.



Facial Recognition Attendance

Upload one classroom photo. SCRFD + MobileFaceNet mark every present student against the student_face table.

`routes/teacher.py`



Admin Dashboard

Bulk-import students via XLSX/CSV, create faculty/mentors, assign classes, and view analytics in one panel.

`routes/admin.py`



Smart Excel Import

No rigid template. Regex auto-detects enrollment numbers, derives batch (e.g. 23BE → 2023-27) and department automatically.

`models/read_excel.py`



Automated Email System

Faculty onboarding emails with credentials + styled OTP password-reset emails (10-min validity) via Gmail SMTP.

`service/mail_service.py`



Four Role Dashboards

Admin, Faculty, Mentor, Student — each with its own template, CSS, and JS bundle. No role-bleed in the UI.

`templates/*.html`



OTP Password Reset

Forgot-password flow emails a one-time password, verifies it server-side, and enforces a 10-minute expiry window.

`routes/forgot.py`

What makes EduSync different.

Most attendance projects stop at a Haar-cascade demo. EduSync ships the production details reviewers actually look for.

Typical attendance project

- Single-face Haar-cascade demo on a webcam feed
- Strict Excel template required for student import
- Whole-image detection — misses back-row faces
- No duplicate handling across overlapping crops
- Manual password resets, no email automation

EduSync EDUSYNC

- ✓ **Image tiling for full-class photos**
512px tiles · 256px overlap
- ✓ **Production-grade SCRFD + MobileFaceNet**
same family as InsightFace buffalo_l
- ✓ **IoU non-max suppression across tiles**
threshold 0.4
- ✓ **Smart auto-detecting Excel parser**
derives batch + dept from enrollment no.
- ✓ **End-to-end email automation**
creds + OTP via Gmail SMTP

"Ships the parts most demos skip — and ships them in 30 commits."

OPEN SOURCE · MIT (CLAIMED)

Try EduSync.

Clone it, run it, and watch a classroom photo become an attendance record.

<> github.com/hetavin/EduSync

🚀 `python app.py → localhost:5000`

👤 Built by Hetavin Pokiya

30

COMMITTS

17 MB

ONNX MODELS IN-REPO

5

MYSQL TABLES

8

FLASK BLUEPRINTS

4

ROLE DASHBOARDS